# P1153R0
# Copying volatile subobjects is not trivial

Published Proposal, 2018-10-04

**Issue Tracking:**
Inline In Spec

**Authors:**
Arthur O'Dwyer
JF Bastien

**Audience:**
EWG

**Project:**
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

**Current Source:**
github.com/Quuxplusone/draft/blob/gh-pages/d1153-volatile-subobjects.bs

**Current:**
rawgit.com/Quuxplusone/draft/gh-pages/d1153-volatile-subobjects.html

---

## Abstract

In C++11, if a class type had both a volatile subobject and defaulted special members, it was trivially copyable. CWG issue 496 (2004–2012) changed this so that if a class type had both a volatile subobject and defaulted special members, it was non-trivially copyable. CWG issues 1746 and 2094 (2013–2016) changed it back, so that if a class type had both a volatile subobject and defaulted special members, it was trivially copyable. This is where we stand today. This paper proposes that class types with volatile subobjects should have their copy and move operations default to deleted. We propose this change because the authors do not know of any correct usage of copying structs with volatile subobjects.

## Table of Contents

# § 1. Motivation for volatile subobjects

Volatile-qualifying a specific non-static data member (NSDM) of a class doesn’t sound like a very useful thing to do. But it does have at least one use, in the embedded-systems niche:

```
struct MemoryMappedRegisters {
    volatile uint32_t r0;
    volatile uint32_t r1;
    volatile uint32_t r2;
    volatile uint32_t r3;
};

MemoryMappedRegisters& g_registers = *(MemoryMappedRegisters*)0x4200;

void foo(MemoryMappedRegisters& regs) {
    ...
    // This access is defined by our implementation to perform a 4-byte
    // followed by two 4-byte loads.
    regs.r0 = 0x1234;
    auto x = regs.r1;
    auto y = regs.r1;  // re-load from the same register
    ...
}
```

This example motivates classes with volatile-qualified NSDMs. It does not motivate *copying* those classes.

In some rare cases the programmer might actually want to copy all of `regs` at once. In that case, the status quo is harmful, rather than helpful. The status quo is to generate a defaulted copy constructor that copies bytes out of the source object in an unspecified manner, ignoring whatever system-specific concerns led the programmer to mark the members `volatile` in the first place.

Further, placing a struct with volatile members on the stack is nonsensical.

# § 2. History

## § 2.1. CWG 496: Is a volatile-qualified type really a POD?

In December 2004, John Maddock writes:

> In 6.9 [basic.types] paragraph 10, the standard makes it quite clear that volatile qualified types are PODs:
>
> > Arithmetic types (6.9.1 [basic.fundamental]), enumeration types, pointer types, and pointer to member types (6.9.2 [basic.compound]), and cv-qualified versions of these types (6.9.3 [basic.type.qualifier]) are collectively called ***scalar types***. Scalar types, POD-struct types, POD-union types (clause 12 [class]), arrays of such types and cv-qualified versions of these types (6.9.3 [basic.type.qualifier]) are collectively called ***POD types***.
>
> However in 6.9 [basic.types] paragraph 3, the standard makes it clear that PODs can be copied "as if" they were a collection of bytes by memcpy:
>
> > For any POD type T, if two pointers to T point to distinct T objects obj1 and obj2, where neither obj1 nor obj2 is a base-class subobject, if the value of obj1 is copied into obj2, using the std::memcpy library function, obj2 shall subsequently hold the same value as obj1.
>
> The problem with this is that a volatile qualified type may need to be copied in a specific way (by copying using only atomic operations on multithreaded platforms, for example) in order to avoid the "memory tearing" that may occur with a byte-by-byte copy.

Proposed resolution, October 2012:

Change 6.9 [basic.types] paragraph 9 as follows:

> ...Scalar types, trivially copyable class types, arrays of such types, and
>
> ~~cv-qualified~~ non-volatile const-qualified versions of these types are collectively called *trivially copyable types*. Scalar types, trivial class types...

Change 10.1.7.1 [dcl.type.cv] paragraphs 6-7 as follows:

> What constitutes an access to an object that has volatile-qualified type is implementation-defined. If an attempt is made to refer to an object defined with a volatile-qualified type through the use of a glvalue with a non-volatile-qualified type, the program behavior is undefined.
>
> [*Note:* `volatile` is a hint to the implementation to avoid aggressive optimization involving the object because the value of the object might be changed by means undetectable by an implementation. Furthermore, for some implementations, `volatile` might indicate that special hardware instructions are required to access the object. See 4.6 [intro.execution] for detailed semantics. In general, the semantics of `volatile` are intended to be the same in C++ as they are in C. —*end note*]

Change 15.8 [class.copy] paragraph 12 as follows:

> A copy/move constructor for class `X` is *trivial* if it is not user-provided, its declared parameter type is the same as if it had been implicitly declared, and if
>
> - class `X` has no virtual functions and no virtual base classes, and
> - class `X` has no non-static data members of volatile-qualified type, and
>
> ...

Change 15.8 [class.copy] paragraph 25 as follows:

> A copy/move assignment operator for class `X` is *trivial* if it is not user-provided, its declared parameter type is the same as if it had been implicitly declared, and if
>
> - class `X` has no virtual functions and no virtual base classes, and
> - class `X` has no non-static data members of volatile-qualified type, and
>
> ...

## § 2.2. CWG 1746: Are volatile scalar types trivially copyable?

In September 2013, Walter Brown writes:

> According to 6.9 [basic.types] paragraph 9,
>
> > Arithmetic types (6.9.1 [basic.fundamental]), enumeration types, pointer types, pointer to member types (6.9.2 [basic.compound]), std::nullptr_t, and cv-qualified versions of these types (6.9.3 [basic.type.qualifier]) are collectively called *scalar types*... Scalar types, trivially copyable class types (Clause 12 [class]), arrays of such types, and non-volatile const-qualified versions of these types (6.9.3 [basic.type.qualifier]) are collectively called *trivially copyable types*.
>
> This is confusing, because “scalar types” include volatile-qualified types, but the intent of the definition of “trivially copyable type” appears to be to exclude volatile-qualified types. Perhaps the second quoted sentence should read something like,
>
> > A non-volatile type `T` or an array of such `T` is called a *trivially copyable type* if `T` is either a scalar type or a trivially copyable class type.
>
> (Note that the following sentence, defining “trivial type,” has a similar formal issue, although it has no actual significance because all cv-qualifiers are permitted.)

Proposed resolution, January 2014:

Change 6.9 [basic.types] paragraph 10 as follows:

> ... ~~Scalar~~ Cv-unqualified scalar types, trivially copyable class types, arrays of such types, and non-volatile const-qualified versions of these types are collectively called *trivially copyable types*...

## § 2.3. CWG 2094: Trivial move/copy constructor for class with volatile member

In March 2015, Daveed Vandevoorde writes:

> The resolution of issue 496 included the addition of 15.8 [class.copy] paragraph 25.2, making a class's copy/move constructor non-trivial if it has a non-static data member of volatile-qualified type. This change breaks the IA-64 ABI, so it has been requested that CWG reconsider this aspect of the resolution.
>
> On a related note, the resolution of issue 496 also changed 6.9 [basic.types] paragraph 9, which makes volatile-qualified scalar types "trivial" but not "trivially copyable." It is not clear why there is a distinction made here; the only actual use of "trivial type" in the Standard appears to be in the description of `qsort`, which should probably use "trivially copyable." (See also issue 1746.)

Notes from the February 2016 meeting:

> CWG agreed with the suggested direction for the changes in 15.8 [class.copy]; the use of "trivial" will be dealt with separately and not as part of the resolution of this issue.

Proposed resolution, June 2016:

Change 6.9 [basic.types] paragraph 9 as follows:

> ...called *POD types*. Cv-unqualified scalar types, trivially copyable class types, arrays of such types, and ~~non-volatile const-qualified~~ cv-qualified versions of these types are collectively called *trivially copyable types*. Scalar types...

Delete bullet 12.2 of 15.8 [class.copy]:

> A copy/move constructor for class `X` is *trivial* if it is not user-provided, its parameter-type-list is equivalent to the parameter-type-list of an implicit declaration, and if
>
> ...
>
> - ~~class `X` has no non-static data members of volatile-qualified type, and~~
>
> ...

Delete bullet 25.2 of 15.8 [class.copy]:

> A copy/move assignment operator for class `X` is *trivial* if it is not user-provided, its parameter-type-list is equivalent to the parameter-type-list of an implicit declaration, and if
>
> ...
>
> - ~~class `X` has no non-static data members of volatile-qualified type, and~~

## § 3. The problem

Library vendors use `std::is_trivially_copyable` to detect object types that can be copied via `memcpy`/`memmove`. This idiom does not work when volatile NSDMs are in play.

```
#include <algorithm>
struct S {
    volatile int i;
};
void foo(S *dst, S *src, int n) {
    std::copy_n(src, n, dst);
}
```

Today, both libc++ and libstdc++ generate a single `memmove` to perform the copy of `src` to `dst`, even though the copying is happening between many discrete (sub)objects that are each volatile-qualified.

Library vendors also have no incentive to change their behavior here; it seems to be strictly non-conforming, but the optimization for *non*-volatile NSDMs is too valuable to give up, and there is currently no way for a library to detect the presence of volatile NSDMs nested within a class.

## § 4. The proposed solution

Whereas

- the current semantics by which volatile NSDMs are copied (trivially) are untenable for library vendors,
- [CWG496] and [CWG2094] show there is no consensus to *change* the semantics by which volatile NSDMs are copied (to make it non-trivial),
- nobody has yet presented a use-case for wanting to copy volatile NSDMs at all,

we propose that

- volatile NSDMs should not be copyable.

That is, the presence of a `volatile`-qualified NSDM in a class should cause the class's copy constructor and copy assignment operator to be defaulted as deleted. The library would continue to detect a Rule-of-Zero-following, volatile-NSDM-having class as `is_trivially_copyable`; but it would also detect it as `not is_copy_constructible` and `not is_move_constructible`, so it wouldn't try to `memcpy` its bytes, and so our tearing problem would be solved.

This solution does not change any ABI, because it merely removes (useless, dangerous) functions that were generated before. In particular, this proposal preserves the *trivial copyability* of structs with volatile members, so that it preserves the calling convention by which they may be returned in registers on IA64.

> ISSUE 1 Hmm, I now think this is wrong. Trivial copyability ([class.prop]/1) requires at least one non-deleted copy-or-move constructor-or-assignment-operator; and our struct will no longer have that. Also, tcanens points to special wording in [class.temporary]/3 that permits returning our struct in registers only if it has a non-deleted copy-or-move constructor. Specifically:
>
> ```
> struct S { volatile int i; };
> S foo() { return S{42}; }  // returns in %eax today
> ```
>
> Whereas if we delete all the constructors:
>
> ```
> struct S { volatile int i; S(const S&) = delete; };
> S foo() { return S{42}; }  // returns on the stack today
> ```
>
> This means we might have no choice but to break ABI.

## § 5. Proposed wording

The wording in this section is relative to WG21 draft N4750, that is, the current draft of the C++17 standard. We quote many unchanged passages here for reference.

[basic.types] #3 is unchanged:

> For any trivially copyable type `T`, if two pointers to `T` point to distinct `T` objects `obj1` and `obj2`, where neither `obj1` nor `obj2` is a potentially-overlapping subobject, if the underlying bytes making up `obj1` are copied into `obj2`, `obj2` shall subsequently hold the same value as `obj1`.

[basic.types] #9 is unchanged (after editorial clarification [PR2255]):

> Arithmetic types, enumeration types, pointer types, pointer-to-member types, `std::nullptr_t`, and cv-qualified versions of these types are collectively called *scalar types*. Scalar types, trivially copyable class types, arrays of such types, and cv-qualified versions of these types are collectively called *trivially copyable types*. Scalar types, trivial class types, arrays of such types and cv-qualified versions of these types are collectively called *trivial types*. Scalar types, standard-layout class types, arrays of such types and cv-qualified versions of these types are collectively called *standard-layout types*.

[class.prop] #1 is unchanged:

> A *trivially copyable class* is a class:
>
> - where each copy constructor, move constructor, copy assignment operator, and move assignment operator is either deleted or trivial,
> - that has at least one non-deleted copy constructor, move constructor, copy assignment operator, or move assignment operator, and
> - that has a trivial, non-deleted destructor.

[class.temporary] #3 is unchanged:

> When an object of class type `X` is passed to or returned from a function, if each copy constructor, move constructor, and destructor of `X` is either trivial or deleted, and `X` has at least one non-deleted copy or move constructor, implementations are permitted to create a temporary object to hold the function parameter or result object. The temporary object is constructed from the function argument or return value, respectively, and the function's parameter or return object is initialized as if by using the non-deleted trivial constructor to copy the temporary (even if that constructor is inaccessible or would not be selected by overload resolution to perform a copy or move of the object). [*Note:* This latitude is granted to allow objects of class type to be passed to or returned from functions in registers. —*end note*]

[class.copy.ctor] #6 is unchanged:

> If the class definition does not explicitly declare a copy constructor, a non-explicit one is declared *implicitly*. If the class definition declares a move constructor or move assignment operator, the implicitly declared copy constructor is defined as deleted; otherwise, it is defined as defaulted. The latter case is deprecated if the class has a user-declared copy assignment operator or a user-declared destructor.

[class.copy.ctor] #7 is unchanged:

> The implicitly-declared copy constructor for a class `X` will have the form
>
> ```
> X::X(const X&)
> ```
>
> if each potentially constructed subobject of a class type `M` (or array thereof) has a copy constructor whose first parameter is of type `const M&` or `const volatile M&`. Otherwise, the implicitly-declared copy constructor will have the form
>
> ```
> X::X(X&)
> ```

[class.copy.ctor] #8 is unchanged:

> If the definition of a class `X` does not explicitly declare a move constructor, a non-explicit one will be implicitly declared as defaulted if and only if
>
> - `X` does not have a user-declared copy constructor,
> - `X` does not have a user-declared copy assignment operator,
> - `X` does not have a user-declared move assignment operator, and
> - `X` does not have a user-declared destructor.
>
> [*Note:* When the move constructor is not implicitly declared or explicitly supplied, expressions that otherwise would have invoked the move constructor may instead invoke a copy constructor. *—end note*]

Change [class.copy.ctor] #10:

> An implicitly-declared copy/move constructor is an inline public member of its class. A defaulted copy/move constructor for a class `X` is defined as deleted if `X` has:
>
> - a potentially constructed subobject type `M` (or array thereof) that cannot be copied/moved because overload resolution, as applied to find `M`'s corresponding constructor, results in an ambiguity or a function that is deleted or inaccessible from the defaulted constructor,
> - a variant member whose corresponding constructor as selected by overload resolution is non-trivial,
> - a non-static data member of `volatile` type (or array thereof), or
> - any potentially constructed subobject of a type with a destructor that is deleted or inaccessible from the defaulted constructor, or,
> - for the copy constructor, a non-static data member of rvalue reference type.
>
> A defaulted move constructor that is defined as deleted is ignored by overload resolution. [*Note:* A deleted move constructor would otherwise interfere with initialization from an rvalue which can use the copy constructor instead. —*end note*]

[class.copy.ctor] #11 is unchanged:

> A copy/move constructor for class `X` is *trivial* if it is not user-provided and if:
>
> - class X has no virtual functions and no virtual base classes, and
> - the constructor selected to copy/move each direct base class subobject is trivial, and
> - for each non-static data member of `X` that is of class type (or array thereof), the constructor selected to copy/move that member is trivial;
>
> otherwise the copy/move constructor is *non-trivial*.

[class.copy.assign] #2 is unchanged:

> If the class definition does not explicitly declare a copy assignment operator, one is declared *implicitly*. If the class definition declares a move constructor or move assignment operator, the implicitly declared copy assignment operator is defined as deleted; otherwise, it is defined as defaulted. The latter case is deprecated if the class has a user-declared copy constructor or a user-declared destructor.
>
> The implicitly-declared copy assignment operator for a class `X` will have the form
>
> ```
> X& X::operator=(const X&)
> ```
>
> if
>
> - each direct base class `B` of `X` has a copy assignment operator whose parameter is of type `const B&`, `const volatile B&`, or `B`, and
> - for all the non-static data members of `X` that are of a class type `M` (or array thereof), each such class type has a copy assignment operator whose parameter is of type `const M&`, `const volatile M&`, or `M`.
>
> Otherwise, the implicitly-declared copy assignment operator will have the form
>
> ```
> X& X::operator=(X&)
> ```

[class.copy.assign] #4 is unchanged:

> If the definition of a class `X` does not explicitly declare a move assignment operator, one will be implicitly declared as defaulted if and only if
>
> - `X` does not have a user-declared copy constructor,
> - `X` does not have a user-declared move constructor,
> - `X` does not have a user-declared copy assignment operator, and
> - `X` does not have a user-declared destructor.

Change [class.copy.assign] #7:

> A defaulted copy/move assignment operator for class `X` is defined as deleted if `X` has:
>
> - a variant member with a non-trivial corresponding assignment operator and `X` is a union-like class, or
> - a non-static data member of `const` non-class type (or array thereof), or
> - a non-static data member of `volatile` type (or array thereof), or
> - a non-static data member of reference type, or
> - a direct non-static data member of class type `M` (or array thereof) or a direct base class `M` that cannot be copied/moved because overload resolution, as applied to find `M`'s corresponding assignment operator, results in an ambiguity or a function that is deleted or inaccessible from the defaulted assignment operator.
>
> A defaulted move assignment operator that is defined as deleted is ignored by overload resolution.

[class.copy.assign] #9 is unchanged:

> A copy/move assignment operator for class `X` is *trivial* if it is not user-provided and if:
>
> - class `X` has no virtual functions and no virtual base classes, and
> - the assignment operator selected to copy/move each direct base class subobject is trivial, and
> - for each non-static data member of `X` that is of class type (or array thereof), the assignment operator selected to copy/move that member is trivial;
>
> otherwise the copy/move assignment operator is *non-trivial*.

# § References

## § Normative References

**[N4750]**
Richard Smith. Working Draft, Standard for Programming Language C++. 7 May 2018. URL: https://wg21.link/n4750

## § Informative References


**[CWG1746]**

Walter Brown. Are volatile scalar types trivially copyable?. September 2013–January 2014. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/cwg_defects.html#1746

**[CWG2094]**

Daveed Vandevoorde. Trivial copy/move constructor for class with volatile member. March 2015–June 2016. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/cwg_defects.html#2094

**[CWG496]**

John Maddock. Is a volatile-qualified type really a POD?. December 2004–October 2012. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/cwg_defects.html#496

**[PR2255]**

Arthur O'Dwyer. 'cv-qualified versions of cv-unqualified scalar types' are just 'scalar types'. July 2018. URL: https://github.com/cplusplus/draft/pull/2255


## § Issues Index

ISSUE 1 Hmm, I now think this is wrong. Trivial copyability ([class.prop]/1) requires at least one non-deleted copy-or-move constructor-or-assignment-operator; and our struct will no longer have that. Also, tcanens points to special wording in [class.temporary]/3 that permits returning our struct in registers only if it has a non-deleted copy-or-move constructor. Specifically:

```
struct S { volatile int i; };
S foo() { return S{42}; }  // returns in %eax today
```

Whereas if we delete all the constructors:

```
struct S { volatile int i; S(const S&) = delete; };
S foo() { return S{42}; }  // returns on the stack today
```

This means we might have no choice but to break ABI. ↵